

GAME CLIENT PROGRAMMER PORTFOLIO

박신우 게임 클라이언트 프로그래머

플레이어 입력, 물리 판정, 상태 전환, 네트워크 동기화를 화면에서 자연스럽게 이어지도록 구현하는 클라이언트 개발자입니다.

Unity WebGL

Unreal Engine 5

C++

Blueprint

OpenGL

TCP/IP

Python

Pico2D



게임 경험을 시스템으로 바꾸는 클라이언트 개발자

소개

자료구조 수업에서 연결 리스트의 삽입, 삭제, 검색, 정렬을 직접 구현하며 기본기를 다졌고, 2D 게임 프로그래밍에서 입력 처리, 상태 갱신, 충돌 판정, 화면 출력이 하나의 게임 루프로 연결되는 과정을 경험했습니다. 이후 Unreal Engine 5, Unity WebGL, OpenGL 네트워크 프로젝트를 거치며 플레이어 경험을 안정적으로 구현하는 방향으로 역량을 확장했습니다.

CLIENT

입력 처리, 카메라 흐름, 상호작용, 충돌 판정, 게임 루프

GAMEPLAY

상태머신, 패턴 판정, 스테이지 흐름, 승패 조건

NETWORK

턴 흐름, 송수신 구조, 캐릭터 동기화, 충돌 결과 반영

BUILD

WebGL 빌드, 브라우저 실행 검증, Git/GitHub 기반 협업

이름

박신우

지원 분야

게임 클라이언트 프로그래머

학력

한국공학대학교 게임공학과, 2027년 3월 졸업 예정

연락처

010-7390-1751 · tlsdn0403@gmail.com

GitHub

github.com/tlsdn0403

04

대표 프로젝트

UE5

졸업작품 TPS
협동 게임

1v1

Unity WebGL
보드 플릭 게임

TCP

OpenGL
네트워크
동기화

FlickDom - Unity WebGL 1대1 전략 플릭 보드게임

프로젝트 개요

FlickDom은 손끝으로 디스크를 튕겨 5x5 보드의 칸을 차지하고, 카드가 요구하는 패턴을 완성해 점수를 거루는 1대1 전략 보드게임입니다. 플레이어는 각자 3개의 디스크를 사용하며, Easy, Normal, Hard 카드 난이도에 따라 점수를 획득하고 먼저 10점에 도달하면 승리합니다.

Unity

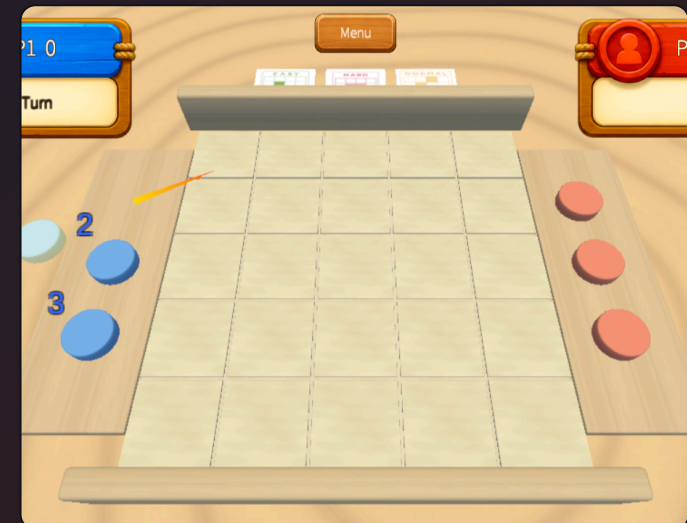
WebGL

1 vs 1 Multiplayer

Board Occupation

Card Pattern Matching

담당 포지션: **Client / Server / Build**. 보드 점령 로직, 카드 패턴 매칭, 턴 **FSM**, 네트워크 구조 수정, **WebGL** 빌드와 배포 검증을 맡았습니다.



팀 역할과 박신우 담당 구현

임찬진

기획 / Shader / Client / Server

게임 규칙, 카드와 점수, 승리 조건, Stage 흐름을 설계하고 Object 선택, Drag/Flick 발사, Trajectory, 카드 획득 애니메이션, Host/Client 동기화 기반을 구현했습니다.

박신우

Client / Server / Build

CLIENT

5x5 Board 점령 Logic, Cell 선택과 배치 후보 처리, Card Pattern Matching, Stage 진행, Monkey 조작과 Camera 흐름, Main Menu 제작

SERVER

FSM 기반 턴 흐름, 멀티플레이 상태 전환, Network 구조 수정

BUILD

WebGL Build와 배포 환경 구성, 브라우저 실행 검증, 최종 Release

황인성

Material / Shader / Art

Disk, Star Asset UV 수정, Board와 그루터기 Material 제작, Triplanar Shader 적용, 보드 소품과 UI Sprite, Victory Image UI 등 게임의 시각 요소를 제작했습니다.

Board

발사 후 충돌 조건을 만족한 디스크를 배치 후보로 처리하고 보드 셀 점령 상태를 갱신했습니다.

Pattern

카드 패턴과 현재 보드 점령 상태를 비교해 점수 획득 여부를 판정했습니다.

Turn FSM

선택, 발사, 정지, 판정, 턴 전환 흐름을 멀티플레이 상태와 연결했습니다.

Release

WebGL 빌드 후 정적 호스팅 환경에서 브라우저 실행과 링크 공유를 검증했습니다.

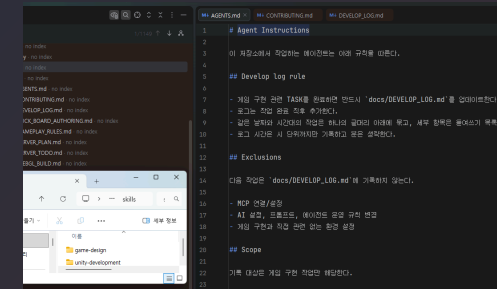
게임 흐름, 멀티플레이, AI 활용 정리



카드 패턴을 완성해 점수를 얻고 먼저 10점에 도달한 플레이어가 승리합니다.



호스트가 방을 생성하고 Join Code를 공유하면 참가자가 같은 방에 접속합니다.



문서 기반 규칙, 서버 계획, WebGL 빌드 조건을 관리하며 구현 흐름을 정리했습니다.

멀티플레이 흐름

- Create Room으로 호스트 방을 만들고 Join Code를 생성합니다.
- 참가자는 코드를 입력해 Join Room으로 접속합니다.
- Players가 2 / 2가 되면 호스트가 Start Game으로 매치를 시작합니다.

AI와 도구 활용

- Codex와 md 문서를 활용해 규칙, 서버 계획, 빌드 조건을 점점했습니다.
- Unity MCP로 Editor 연동 작업을 보조하고, VARCO AI로 사운드와 BGM 제작 흐름을 정리했습니다.
- 최종 결과는 HTML 소개서, 팀 역할 문서, AI 활용 문서로 분리해 발표 자료화했습니다.

감염: 죽음의 도시 - UE5 C++ 좀비 TPS 협동 게임

프로젝트 개요

좀비 바이러스가 퍼진 도시에서 3명이 협동해 1스테이지에서 탈출에 필요한 아이템을 파밍하고, 2스테이지에서 트럭을 타고 전투하며 도시를 탈출하는 졸업작품 프로젝트입니다.

Unreal Engine 5

C++

Blueprint

Collision / Trace

Level Streaming

01

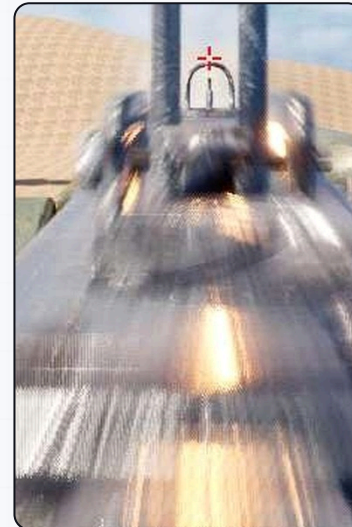
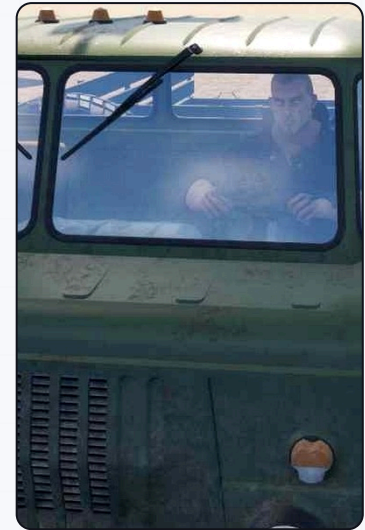
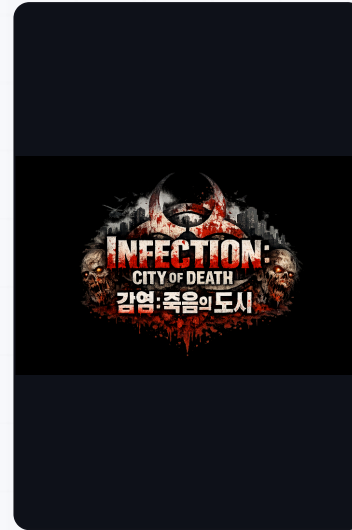
플레이어, 트럭, 아이템 상호작용을 Driver / Cargo / Turret 같은 접근 타입으로 분리했습니다.

02

좀비 피격 위치를 HitResult와 BoneName으로 판정하고 부위별 내구도와 신체 분리 상태로 연결했습니다.

03

Stage2 무한 맵은 동적 로드 멈춤을 줄이기 위해 타일을 미리 생성하고 풀에서 재배포하는 방식으로 개선했습니다.

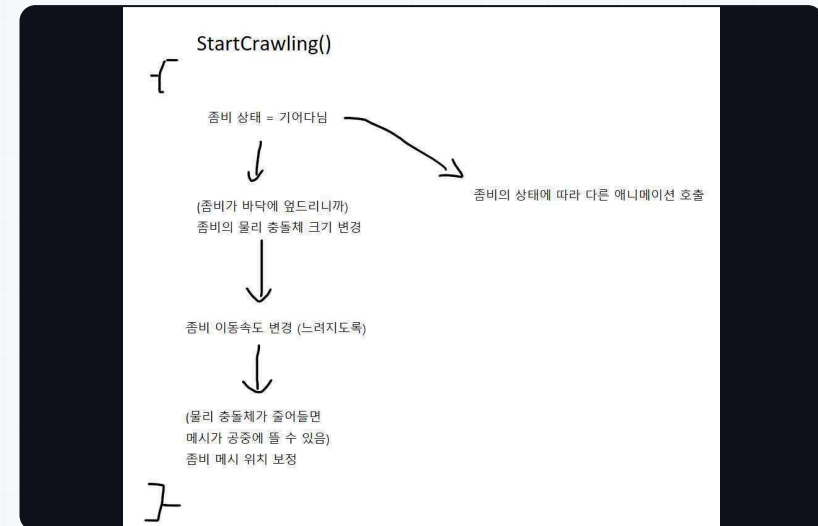


좀비 피격 부위 분리 시스템

구현 구조

- **ApplyPointDamage**에서 **FHitResult**를 넘기고 **OnZombieDamaged**가 실제 피격 위치와 **BoneName**을 기준으로 처리합니다.
- **TMap<FName, float>**로 **head, upperarm, lowerarm, thigh, calf, spine** 등 부위별 내구도를 관리합니다.
- 내구도가 0 이하가 되면 **DismemberLimb**에서 **BreakConstraint, HideBoneByName**, 물리 시뮬레이션, **Impulse**를 적용합니다.
- 다리 계열 뼈가 분리되면 **StartCrawling**으로 전환해 캡슐 크기와 이동 속도를 낮추고 기어가는 좀비 상태로 바꿉니다.

결과 흐름: ApplyPointDamage - OnZombieDamaged - 부위 내구도 감소 - DismemberLimb / StartCrawling



트럭 상호작용과 Stage2 무한 맵 개선



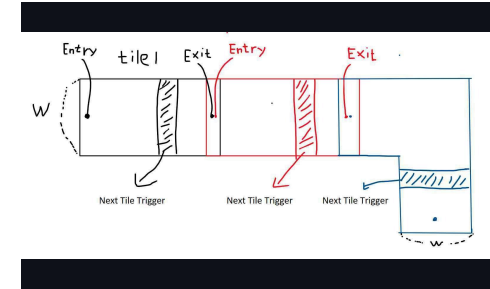
아이템 적재

파밍 아이템을 적재함 보관 상태로 옮겨 Stage1 파밍과 Stage2 이동 흐름이 이어지도록 구성했습니다.



탑승과 무기 사용

적재함 Trigger 범위에서 플레이어가 트럭 위에 탑승하고, 장착된 기관총을 사용할 수 있게 했습니다.



타일 풀 재배포

Next Tile Trigger 순간의 동적 로드 부담을 줄이기 위해 미리 만든 타일을 풀에 보관하고 Exit / Entry 기준으로 이어 붙였습니다.

버그 해결 경험

트럭 짐칸 탑승 중 플레이어가 튀어 오르거나 뒤로 밀리는 문제를 충돌 구조와 이동 기준 문제로 분리해 분석했습니다. 트럭과 플레이어 충돌을 분리한 뒤 SetBase(NewBase)로 트럭 바닥을 이동 기준으로 잡고, 점프와 하차 시 SetBase(nullptr)와 상태 초기화를 명확히 처리해 멀티플레이 환경의 순간 위치 보정 문제를 줄였습니다.

Pico2D Hamtori Escape - Python 기반 2D 탈출 게임

프로젝트 개요

2D 게임 프로그래밍 수업에서 제작한 탈출형 어드벤처 게임입니다. 햄토리가 장애물과 NPC를 피해 스테이지별 탈출구로 이동하며, 특정 사용자가 즐길 수 있는 귀여운 회피형 게임을 목표로 완성했습니다.

Python

Pico2D

State Machine

Behavior Tree

Collision

01

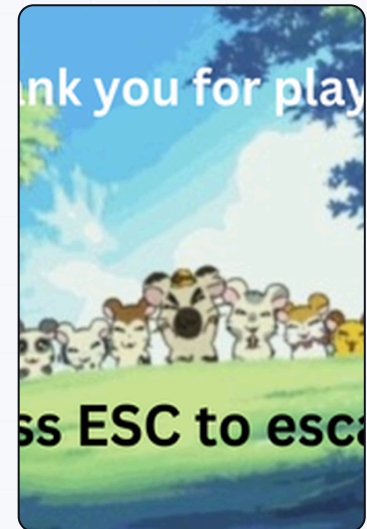
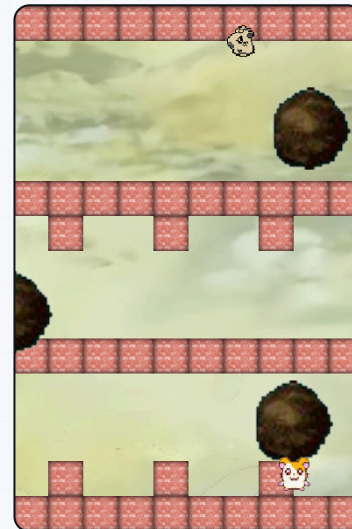
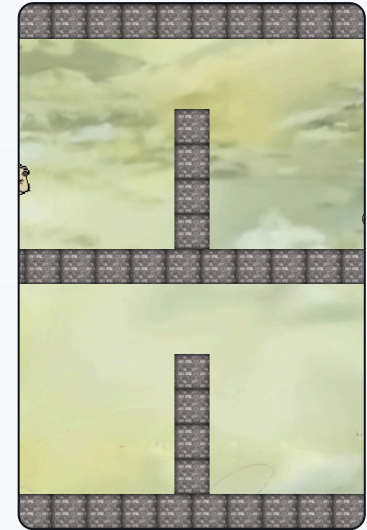
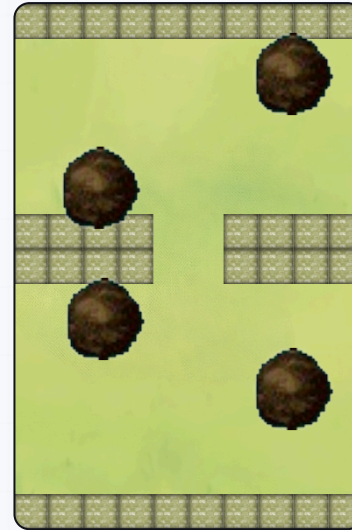
Stage 1: 구르는 돌을 피해 위쪽 탈출구로 이동하는 회피 중심 스테이지를 구성했습니다.

02

Stage 2: NPC의 추격과 배회 패턴을 피해 출구까지 이동하도록 상태 기반 적 행동을 구성했습니다.

03

Stage 3: NPC를 돌에 유도해 제거한 뒤, 돌과 NPC를 피해 최종 탈출하는 흐름으로 난이도를 확장했습니다.



NGP Fall Guys Network - OpenGL 네트워크 동기화

프로젝트 목표

기존 로컬 장애물 경주 게임을 3인 네트워크 플레이가 가능한 구조로 전환한 프로젝트입니다. 서버가 캐릭터와 장애물 상태를 공유하고, 클라이언트가 렌더링과 충돌 처리 결과를 반영하는 흐름을 구현했습니다.

C++

OpenGL

TCP/IP

Server / Client

AABB Collision

01

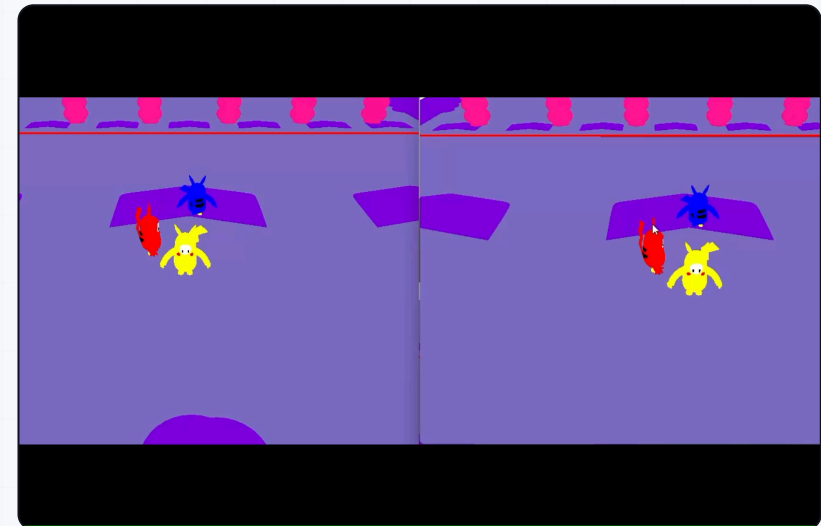
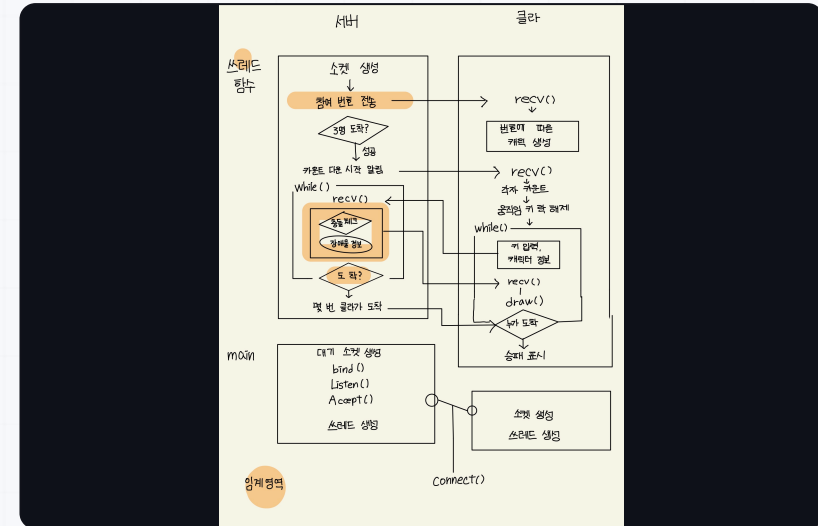
분할 뷰포트 구조를 하나의 클라이언트 화면 흐름으로 병합하고, 접속 순서에 따라 캐릭터 번호를 부여했습니다.

02

C2S_Character와 S2C_Character 경로를 통해 이동 정보를 서버로 보내고 다른 클라이언트에 전달했습니다.

03

캐릭터-캐릭터 및 캐릭터-장애물 충돌을 AABB로 판정하고 서버 상태에 반영했습니다.



문제를 증상보다 구조로 추적하는 방식

기술 문제 해결

졸업작품의 트럭 탑승 버그는 처음에는 Mesh 충돌처럼 보였지만, 실제로는 충돌, 월드 좌표 기준 이동, 움직이는 플랫폼 기준, 멀티플레이 위치 보정이 함께 얹힌 문제였습니다. 충돌 구조를 분리하고 SetBase 적용과 해제 시점을 명확히 하면서, 단일 증상이 아니라 상태 전환 전체를 확인해야 한다는 기준을 세웠습니다.

협업 문제 해결

네트워크 게임 프로젝트에서 선행 리팩터링 지연으로 팀원 간 소통이 끊겼을 때, 개인 감정보다 작업 의존 관계와 진행 상황 공유 문제로 보았습니다. 팀원들을 한자리에 모아 완료 범위와 남은 범위, 대기 업무를 정리했고 이후 졸업작품 팀에서도 불만과 의견을 초기에 공유하는 방식으로 협업을 이어갔습니다.

Observe

화면에 나타난 현상과 실제 시스템 상태를 분리해 확인합니다.

Trace

입력, 충돌, 상태 전환, 네트워크 동기화 순서로 원인을 좁힙니다.

Fix

증상을 가리는 수정이 아니라 기준 상태와 해제 조건을 명확히 합니다.

Share

팀원이 다음 작업을 이어갈 수 있도록 진행 상황과 남은 위험을 공유합니다.

클라이언트 개발자로서의 방향

입력 판정, 물리, 애니메이션, 사운드, 카메라, 네트워크 상태가 서로 따로 움직이지 않고 하나의 플레이 경험으로 느껴지게 만드는 개발자가 되는 것이 목표입니다. 플레이어가 조작한 결과를 즉각적이고 안정적으로 전달하는 시스템 구현에 집중하겠습니다.

프로젝트 링크와 확인 자료

Contact

박신우 · 게임 클라이언트 프로그래머
010-7390-1751 · tlsdn0403@gmail.com
GitHub: github.com/tlsdn0403

FLICKDOM GITHUB

github.com/AACHANJINAA/FlickDom

FLICKDOM WEBGL

fantastic-pothos-5ab193.netlify.app

FLICKDOM VIDEO

youtube.com/watch?v=IR4KDTCyNKK

INFECTION GITHUB

github.com/tlsdn0403/SYJ

HAMTORI GITHUB

github.com/tlsdn0403/2DGP-Project

NGP GITHUB

github.com/tlsdn0403/NGP_Project

